The Journal of Open Source Software

PyMatLib: A Python Library for Temperature-Dependent Material Property Processing in Scientific Simulations

Rahil Miten Doshi [1]

1 Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

PyMatLib is an open-source Python library that revolutionizes how scientists and engineers handle temperature-dependent material properties in computational simulations. When materials are heated or cooled, their physical characteristics like thermal conductivity, density, and heat capacity change significantly. This creates challenges for computer simulations of processes like metal casting, heat treatment, or thermal analysis.

The library provides a simple way to define these temperature-dependent properties using YAML configuration files, then automatically converts them into mathematical expressions that can be used in scientific simulations. PyMatLib supports both pure metals and alloys, handles six different ways of defining properties (from simple constants to complex equations), and automatically manages dependencies between different material properties.

The library has been designed specifically for high-performance simulations in computational materials science and heat transfer applications, integrating seamlessly with existing scientific computing workflows.

Statement of Need

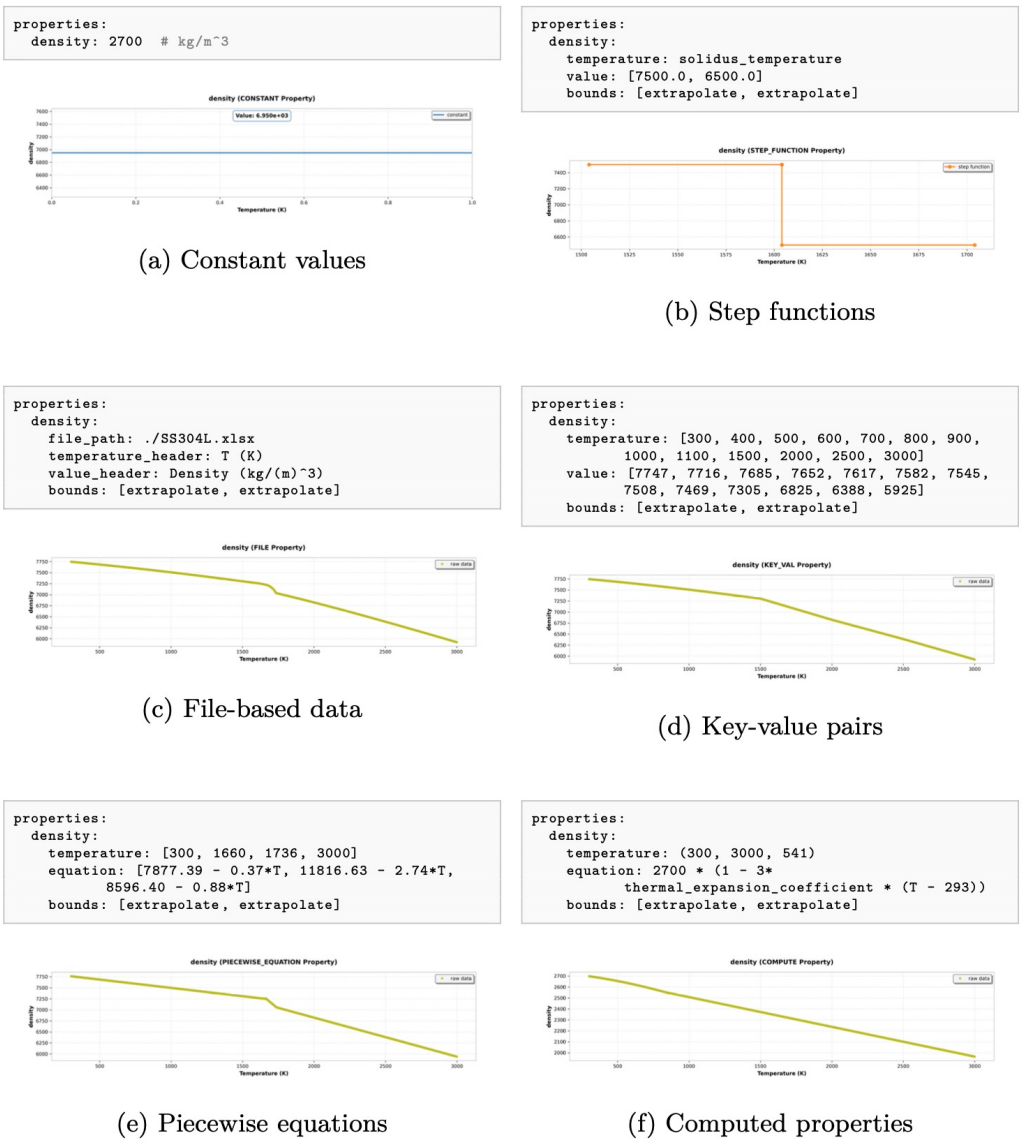
Accurate representation of temperature-dependent material properties is fundamental to computational materials science, thermal analysis, and multi-physics simulations (Lewis et al., 1996; Zienkiewicz et al., 2013). Traditional approaches often require manual interpolation, custom scripting, or proprietary software solutions that lack flexibility and reproducibility (Ashby, 2013). Existing materials databases like NIST (National Institute of Standards and Technology, 2023) and CoolProp (Bell et al., 2014) typically provide tabular data without integrated processing capabilities, forcing researchers to implement ad-hoc solutions for each simulation framework.

While established tools like CoolProp (Bell et al., 2014) and NIST databases (National Institute of Standards and Technology, 2023) provide comprehensive thermodynamic data, they lack integrated symbolic processing and dependency resolution capabilities. CoolProp focuses on fluid properties with limited solid material support and no symbolic integration, while NIST WebBook provides tabular data without processing capabilities or dependency management. CALPHAD databases (Lukas et al., 2007) require specialized software and lack integration with general-purpose simulation codes. Custom implementations often lack validation, reproducibility, and standardized interfaces.

Key Features

PyMatLib addresses critical gaps in material science software by providing a standardized, extensible framework that offers unprecedented flexibility in material property definition through several innovative capabilities:

- **Flexible Input Methods**: The library supports six different property definition methods (constant values, step functions, file-based data, key-value pairs, piecewise equations, and computed properties), allowing users to choose the most appropriate format for their data sources and modeling requirements (Figure 1). File-based data processing leverages pandas (McKinney & others, 2010) for robust handling of various formats including Excel (.xlsx), CSV, and text files with comprehensive missing value detection, data cleaning, duplicate removal, and validation.



(a) Constant values

(b) Step functions

(c) File-based data

(d) Key-value pairs

(e) Piecewise equations

(f) Computed properties

Figure 1: PyMatLib input flexibility demonstration showing different property definition methods: (a) constant values for simple properties, (b) step functions for phase transitions, (c) key-value pairs for experimental data, (d) piecewise equations for complex relationships, and (e) computed properties with automatic dependency resolution.

- **Universal Material Support**: The framework supports both pure metals and alloys through a unified interface, with extensibility for additional material types. Pure metals use melting/boiling temperatures, while alloys use solidus/liquidus temperature ranges.
- **Robust Data Quality Assurance**: Built-in data validation includes duplicate temperature removal, missing value handling with configurable thresholds, automatic data sorting, and support for various file encodings and missing value representations commonly found in experimental datasets. This ensures data integrity and prevents common errors in materials property processing.
- **Optional Data Simplification**: The library supports regression-based data simplification using pwlf (Jekel & Venter, 2019) for large complex datasets, allowing users to reduce computational overhead and memory usage while maintaining accuracy. Piecewise linear fitting with configurable polynomial degrees enables efficient approximation of complex property relationships while preserving essential physical behavior (Figure 2).

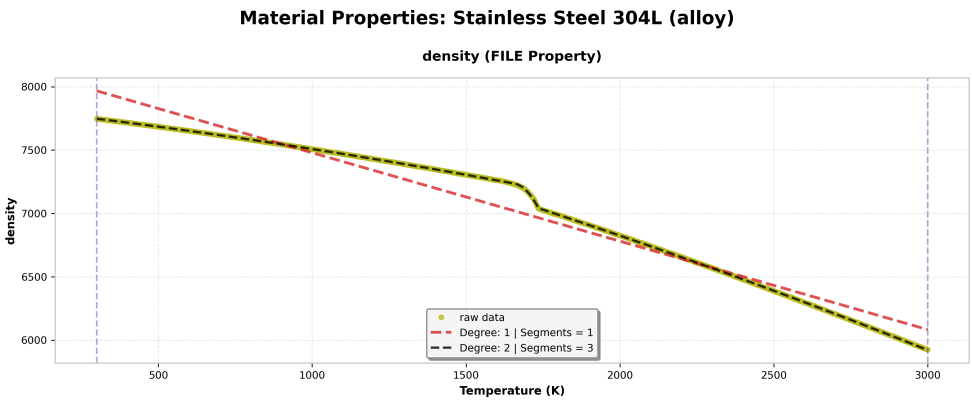


Figure 2: Regression capabilities showing data simplification effects: raw experimental data (points) fitted with different polynomial degrees and segment configurations, demonstrating how PyMatLib can reduce complexity while maintaining physical accuracy.

- **Intelligent Simplification Timing**: PyMatLib provides sophisticated control over when data simplification occurs in the dependency chain through the `simplify` parameter (`pre` or `post`). When set to `pre`, properties are simplified before being passed to dependent properties, optimizing computational performance for complex dependency networks. When set to `post`, the raw piecewise function with all data points is preserved during dependency resolution, with simplification applied only after all dependent calculations are complete. This ensures maximum accuracy in interdependent property calculations while still providing the benefits of data simplification. This timing control allows users to balance computational efficiency with numerical accuracy based on their specific simulation requirements. Here is an example configuration for regression and simplification options in PyMatLib:

```
regression:      # Optional regression configuration
  simplify: pre  # 'pre' (before processing) or 'post' (after processing)
  degree: 2      # Polynomial degree for regression
  segments: 3    # Number of piecewise segments
```

- **Configurable Boundary Behavior**: Users can specify how properties behave outside defined temperature ranges, choosing between constant extrapolation or linear extrapolation based on their physical understanding of the material (Figure 3).

```
bounds: [constant, extrapolate]  # Boundary behavior: 'constant' or 'extrapolate'
```

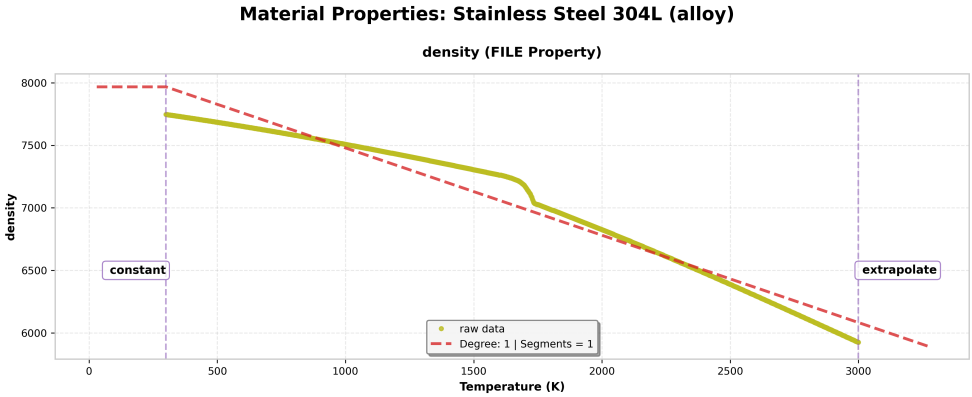


Figure 3: Boundary behavior options in PyMatLib showing the same thermal conductivity property with different extrapolation settings: constant boundaries (left) maintain edge values outside the defined range, while extrapolate boundaries (right) use linear extrapolation.

- **Automatic Dependency Resolution**: Intelligent processing order determination for computed properties ensures mathematical dependencies are resolved correctly without manual intervention. The library automatically detects circular dependencies and provides clear error messages for invalid configurations.
- **Bidirectional Property-Temperature Conversion**: The library automatically generates inverse piecewise functions enabling temperature = f(property) calculations alongside the standard property = f(temperature) relationships. This bidirectional capability is essential for energy-based numerical methods (Voller & Prakash, 1987), phase-change simulations, and iterative solvers where temperature must be determined from known property values. The inverse function generation supports linear piecewise segments (either through default linear interpolation or explicit degree=1 regression), ensuring robust mathematical invertibility.
- **Built-in Validation Framework**: Built-in validation ensures YAML configuration correctness, including composition sum verification, required field checking for pure metals versus alloys, property name validation, and regression parameter bounds checking. This prevents common configuration errors and ensures reproducible material definitions (Roache, 1998).
- **Integrated Visualization**: Automatic plot generation enables users to verify their material definitions visually, with the option to disable visualization for production workflows after validation.

```python
import sympy as sp
from pymatlib.parsing.api import create_material

# Create a material with symbolic temperature and enable plotting
T = sp.Symbol('T')
material_T = create_material('path/to/material.yaml', T, enable_plotting=True)
```

Unlike existing tools, PyMatLib uniquely combines symbolic mathematics (Meurer et al., 2017), automatic dependency resolution, and seamless integration with scientific computing workflows (Harris et al., 2020; Hunter, 2007). The library integrates directly with simulation frameworks like pystencils (Bauer et al., 2019) and waLBerla (Bauer et al., 2021) for high-performance computing applications. PyMatLib is open-source, distributed under the BSD-3-Clause license. Code and documentation to use this package are available on GitHub.

DRAFT

Example YAML Configuration

PyMatLib uses a YAML-based configuration system for defining material properties, allowing users to specify properties in a human-readable format. The YAML files can include pure metals with melting/boiling temperatures or alloys with solidus/liquidus temperature ranges.

Pure Metal Example

```yaml
# ================================================
# PYMATLIB MATERIAL CONFIGURATION FILE - PURE METAL
# ================================================

name: Aluminum
material_type: pure_metal

# Composition must sum to 1.0 (for pure metals, single element = 1.0)
composition:
  Al: 1.0  # Aluminum

# Required temperature properties for pure metals
melting_temperature: 933.47  # Solid becomes liquid (K)
boiling_temperature: 2743.0  # Liquid becomes gas (K)

properties:
    density:
        temperature: (300, 3000, 541)
        equation: 2700 * (1 - 3*thermal_expansion_coefficient * (T - 293))
        bounds: [constant, constant]
```

Alloy Example

```yaml
# ==========================================
# PYMATLIB MATERIAL CONFIGURATION FILE - ALLOY
# ==========================================

name: Stainless Steel 304L
material_type: alloy

# Composition fractions must sum to 1.0
composition:
Fe: 0.675  # Iron
Cr: 0.170  # Chromium
Ni: 0.120  # Nickel
Mo: 0.025  # Molybdenum
Mn: 0.010  # Manganese

# Required temperature properties for alloys
solidus_temperature: 1605.          # Melting begins (K)
liquidus_temperature: 1735.         # Melting is completely melted (K)
initial_boiling_temperature: 3090.  # Boiling begins (K)
final_boiling_temperature: 3200.    # Material is completely vaporized (K)

properties:
    density:
        file_path: ./SS304L.xlsx
```

DRAFT

```yaml
temperature_header: T (K)
value_header: Density (kg/(m)^3)
bounds: [constant, extrapolate]
regression:  # Optional regression configuration
    simplify: pre  # Simplify before processing
    degree: 2      # Use quadratic regression for simplification
    segments: 3    # Fit with 3 segments for piecewise linear approximation
```

Complete YAML configurations are provided in the PyMatLib documentation for both pure metals and alloys.

Research Applications

The library can be applied in research projects involving alloy design and optimization with accurate representation of solidus-liquidus temperature ranges (Callister & Rethwisch, 2018), energy-based finite element methods for thermal analysis (Hughes, 2012), multiscale simulations linking molecular dynamics with continuum mechanics (Tadmor & Miller, 2011), and high-performance computing applications in computational fluid dynamics and heat transfer modeling. Its YAML-based configuration system, powered by ruamel.yaml parsing (Wel, 2023), supports the FAIR principles (Wilkinson et al., 2016) for scientific data management, enabling reproducible research across different simulation codes and research groups.

Comparison with Existing Tools

Feature	PyMatLib	CoolProp	NIST WebBook	Custom Scripts	Materials Project
Unique Combinations					
Symbolic Integration	Yes	No	No	Limited	No
Dependency Resolution	Yes	No	No	Manual	No
Multiple Input Methods	Yes (6 types)	No	No	Variable	No
Material Support					
Solid Materials Focus	Yes	Limited	Yes	Variable	Yes
Custom Property Definition	Yes	No	No	Yes	No
Temperature Dependencies	Yes	Yes	Yes	Variable	Limited
Scientific Rigor					
Data Validation	Yes	Yes	No	Variable	Yes
Reproducibility	Yes	Yes	Limited	Variable	Yes
Open Source	Yes	Yes	No	Yes	Yes

Key Advantage: PyMatLib is the only tool that combines symbolic mathematics with intelligent dependency resolution, enabling seamless integration with simulation frameworks while maintaining scientific reproducibility.

Acknowledgements

The development of PyMatLib was supported by the Friedrich-Alexander-Universität Erlangen-Nürnberg. We acknowledge the contributions of the SymPy (Meurer et al., 2017), NumPy

DRAFT

(Harris et al., 2020), pandas (McKinney & others, 2010), matplotlib (Hunter, 2007), and ruamel.yaml (Wel, 2023) development communities, whose libraries provide the symbolic mathematics, numerical computing, data processing, visualization, and configuration parsing capabilities that form the foundation of PyMatLib.

References

Ashby, M. F. (2013). *Materials and design: The art and science of material selection in product design* (3rd ed.). Butterworth-Heinemann. ISBN: 978-0080982052

Bauer, M., Hötzer, J., Ernst, D., Hammer, J., Seiz, M., Hierl, H., Hönig, J., Köstler, H., Nestler, B., & Rüde, U. (2019). Code generation for massively parallel phase-field simulations. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–12. https://doi.org/10.1145/3295500.3356186

Bauer, M., Köstler, H., & Rüde, U. (2021). waLBerla: A block-structured high-performance framework for multiphysics simulations. *Computers & Mathematics with Applications*, *81*, 478–501. https://doi.org/10.1016/j.camwa.2020.01.007

Bell, I. H., Wronski, J., Quoilin, S., & Lemort, V. (2014). CoolProp: An open-source reference-quality thermophysical property library. *Industrial & Engineering Chemistry Research*, *53*(6), 2498–2508. https://doi.org/10.1021/ie4033999

Callister, W. D., & Rethwisch, D. G. (2018). *Materials science and engineering: An introduction* (10th ed.). John Wiley & Sons. ISBN: 978-1119405498

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hughes, T. J. R. (2012). *The finite element method: Linear static and dynamic finite element analysis*. Dover Publications. ISBN: 978-0486411811

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Jekel, C. F., & Venter, G. (2019). Pwlf: A python library for fitting 1D continuous piecewise linear functions. *Python in Science Conference*. https://doi.org/10.25080/Majora-7ddc1dd1-019

Lewis, R. W., Morgan, K., Thomas, H. R., & Seetharamu, K. N. (1996). *Finite element analysis of heat transfer and fluid flow*. John Wiley & Sons. ISBN: 978-0471943617

Lukas, H. L., Fries, S. G., & Sundman, B. (2007). *Computational thermodynamics: The calphad method*. Cambridge University Press. ISBN: 978-0521868112

McKinney, W., & others. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, *445*, 51–56. https://doi.org/10.25080/Majora-92bf1922-00a

Meurer, A., Smith, C. P., Paprocki, M., Čertík, O., Kirpichev, S. B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J. K., Singh, S., Rathnayake, T., Vig, S., Granger, B. E., Muller, R. P., Bonazzi, F., Gupta, H., Vats, S., Johansson, F., Pedregosa, F., … Scopatz, A. (2017). SymPy: Symbolic computing in python. *PeerJ Computer Science*, *3*, e103. https://doi.org/10.7717/peerj-cs.103

National Institute of Standards and Technology. (2023). *NIST chemistry WebBook*. https://webbook.nist.gov/chemistry/

DRAFT

Roache, P. J. (1998). *Verification and validation in computational science and engineering*. Hermosa Publishers. ISBN: 978-0913478080

Tadmor, E. B., & Miller, R. E. (2011). *Modeling materials: Continuum, atomistic and multiscale techniques*. Cambridge University Press. ISBN: 978-0521856980

Voller, V. R., & Prakash, C. (1987). A fixed grid numerical modelling methodology for convection-diffusion mushy region phase-change problems. *International Journal of Heat and Mass Transfer*, *30*(8), 1709–1719. https://doi.org/10.1016/0017-9310(87)90317-6

Wel, A. van der. (2023). *Ruamel.yaml: YAML 1.2 loader/dumper package for python*. https://yaml.readthedocs.io/

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L. B. da, Bourne, P. E., & others. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*(1), 1–9. https://doi.org/10.1038/sdata.2016.18

Zienkiewicz, O. C., Taylor, R. L., & Zhu, J. Z. (2013). *The finite element method: Its basis and fundamentals* (7th ed.). Butterworth-Heinemann. ISBN: 978-1856176330

DRAFT